| Problem Chosen | 2026 MCM/ICM Summary Sheet | Team Control Number |
|---|---|---|
| C | | 2625838 |

# Summary Sheet

**Team #M2026232**

We study *Dancing with the Stars* (DWTS) as an inverse problem: judges' scores and weekly eliminations are observed across 34 seasons, but audience vote totals are withheld. The key unknown is each active contestant's weekly fan vote share, which drives eliminations under different aggregation rules.

## What we built

- A reproducible season–week–contestant panel that standardizes active sets, elimination labels, and judge-score aggregates under irregular show formats (no-elimination weeks, multi-elimination weeks, varying numbers of judges, and post-exit "zero" encodings).
- A convex inference model (Task A) that reconstructs latent fan vote shares consistent with observed eliminations, and an identifiability measure (KPI2) quantifying how uniquely votes are determined each week.
- A replay engine (Tasks B/C) that re-simulates every season under percent-based versus rank-based aggregation (optionally with a bottom-two judges' save), enabling controlled counterfactual comparisons and controversy case studies.
- Mixed-effects attribution models (Task D1) separating drivers of judges' scores and fan support, and a simple, interpretable new voting rule (Task D2) stress-tested by replay and uncertainty perturbations.

## Sanity-check evidence (data integrity)

From the weekly panel we define *eligible* season–weeks (including multi-elimination weeks evaluated by bottom-$k$ set match). Under a BL-0 baseline (judge-only with a uniform fan proxy), the elimination hit-rate is 0.364 (rank aggregation) and 0.375 (percent aggregation) across 264 eligible weeks, and the two schemes disagree on the predicted eliminated set in 3.8% of eligible weeks (FlipRate = 0.038). These metrics validate consistent labeling and replay mechanics and serve as a baseline—not the final inferred-vote results.

## Key deliverables for decision-making

- Season-by-season comparisons of how often rank vs. percent rules change eliminations and winners, and when judges' save overturns vote-driven outcomes.
- Uncertainty reports showing which weeks/contestants are non-identifiable from observed eliminations alone.
- A recommended voting system with two intuitive control knobs (judge–fan balance and popularity compression), selected via multi-metric stress tests to improve stability without sacrificing interpretability.

## Management recommendation

Pilot the proposed rule in a one-season "shadow mode" (computed in parallel with the broadcast rule) and adopt a parameter region that preserves replay fidelity on eligible weeks while improving stability under vote-uncertainty perturbations.

# Contents

# 1 Introduction

*Dancing With The Stars* (DWTS) combines judges' scores with audience voting to eliminate contestants weekly, yet the audience vote totals remain confidential.[1] This creates a recurring tension between "technical merit" and "popular support," amplified by format changes across seasons (percent-based versus rank-based aggregation, and the later bottom-two judges' save). The dataset [2] spans 34 seasons with contestant attributes, weekly judges' scores, and realized eliminations, but it does not reveal audience votes—turning the central signal of fan preference into a latent variable.

**Our key methodological idea is to treat DWTS as a replayable inverse problem.** We first construct a canonical season–week–contestant panel with an explicit active set (excluding post-exit zero-score records and flagging special episodes). We then infer weekly fan vote *shares* by solving a constrained optimization problem: elimination-consistency constraints encode the show's rule mechanics, slack variables absorb exceptional weeks, and a temporal smoothness prior selects a stable solution among many feasible explanations. With inferred fan support in hand, we implement a rule replay engine that simulates eliminations week-by-week under alternative aggregation schemes (rank-based vs. percent-based, with optional judges' save), enabling direct counterfactual comparisons. The inference module outputs not only point estimates $\hat{p}_{i,t}$ but also uncertainty summaries (e.g., feasible ranges and sensitivity margins). These outputs become direct inputs to the replay engine, closing the loop from latent-vote inference to counterfactual elimination paths and to quantitative evidence for rule comparison and new-system validation.

**We evaluate models and voting rules using replay-based metrics that align with the show's decisions.** For vote inference, we report elimination-consistency rates, feasible-range widths (as an identifiability proxy), and sensitivity to regularization strength. For rule comparisons and new-system design, we quantify outcome divergence (who would be eliminated when), shifts in finalist composition, and robustness under stress tests that vary vote weights and the assumed rule-switch season.

**To ensure interpretability, we benchmark against simple baselines** (e.g., judges-only elimination and naive fan-share models) and require that any added complexity yields measurable improvements in replay fidelity and stability. Building on inferred votes, we fit mixed-effects models to separately explain judges' scoring and fan preference, isolating professional-dancer effects and celebrity attributes, and we use these insights to design a parameterized voting system that balances fairness, suspense, and robustness.

The remainder of the paper is organized as follows: Section 2 formalizes tasks, assumptions, and notation; Section 3 describes data processing and feature construction; Sections 4–7 present vote inference, rule replay, mechanism analysis, and the proposed voting system with robustness tests; Section 8 concludes; and Section 9 provides a memo to management.

# 2 Problem Decomposition and Analysis

**This section formalizes DWTS as a replayable inverse problem and specifies the modeling objects that close our pipeline.** We define the weekly active set, the observable judges' score signal, and the realized elimination outcomes, then introduce the latent fan vote share $p_{i,t}$ as the primary unknown. Our inference module outputs point estimates $\hat{p}_{i,t}$ together with uncertainty summaries, which feed a unified rule replay engine; this engine generates counterfactual elimination paths used for rule comparison, case-based attribution, and the validation of a proposed new voting system.

## 2.1 Problem Decomposition

**We decompose the prompt into four linked tasks with shared inputs and a common replay-based evaluation layer.**

- **Task A: Fan vote inference with uncertainty. Input:** weekly judges' scores and the realized eliminated

set; **Output:** weekly fan vote shares $p_{i,t}$ (and uncertainty bands) over the active set; **Validation:** elimination-consistency under the assumed rule, plus identifiability diagnostics (feasible-range widths and margin-based sensitivity).

- **Task B: Voting-rule comparison via replay. Input:** judges' scores and inferred $p_{i,t}$; **Output:** counterfactual week-by-week eliminations under percent-based vs. rank-based aggregation (and variants); **Validation:** season-level and week-level outcome divergence measures and "fan bias" indicators.
- **Task C: Controversial contestants and judges' save attribution. Input:** inferred $p_{i,t}$ and replay engine; **Output:** case-study narratives supported by counterfactual trajectories with/without bottom-two judges' save; **Validation:** whether the mechanism explains observed controversies and how sensitive conclusions are to vote uncertainty.
- **Task D: Mechanism analysis and new system design. Input:** panel features (celebrity attributes, professional dancer identifiers) and inferred votes; **Output:** mixed-effects estimates for drivers of judges' scoring versus fan preference, and a parameterized voting rule; **Validation:** historical replay, robustness checks, and sensitivity analysis over hyperparameters and rule-switch assumptions.

## 2.2 Key Assumptions

We adopt a small, testable assumption budget to make the inverse problem well-posed while keeping conclusions falsifiable.

1. **Share normalization.** Fan votes are modeled as within-week shares over the active set.
2. **Rule-governed elimination.** Eliminations follow the documented aggregation rule (percent-based or rank-based), optionally augmented by bottom-two judges' save where applicable.
3. **Active-set definition and special episodes.** Post-exit "zeros" represent inactivity and are excluded from the active set; special episodes are explicitly flagged and may be accommodated via slack rather than forced exact compliance.
4. **Judges as observed signal.** Judges' scores provide an observed performance signal taken as exogenous to the voting mechanism.
5. **Temporal smoothness.** Fan preference evolves smoothly week-to-week, motivating temporal regularization.

The season at which the show returns to rank-based aggregation is treated as an explicit assumption and is stress-tested using adjacent cutoffs.

## 2.3 Key Notations

Table 1 summarizes indices, sets, observed quantities, latent fan vote shares, and rule-dependent combined scores used throughout the inference and replay pipeline.

| Symbol | Definition |
|---|---|
| $s$ | Season index, $s = 1, \ldots, 34$ |
| $t$ | Week index within season $s$ |
| $i$ | Contestant (celebrity–pro pair) index |
| $A_{s,t}$ | Active contestants in season $s$, week $t$ |
| $E_{s,t}$ | Eliminated set at end of week $t$; $m_{s,t} = \lvert E_{s,t}\rvert$ |
| $j_{i,t,k}$ | Score given by judge $k$ to contestant $i$ in week $t$ |
| $J_{i,t}$ | Total judges' score: $J_{i,t} = \sum_k j_{i,t,k}$ (skip missing) |
| $q^J_{i,t}$ | Judges' share: $q^J_{i,t} = J_{i,t} \big/ \sum_{r \in A_{s,t}} J_{r,t}$ |
| $p_{i,t}$ | Fan vote share (latent), $\sum_{i \in A_{s,t}} p_{i,t} = 1$ |
| $\alpha$ | Weight on judges under percent rule (default $\alpha = 0.5$; stress-tested) |
| $S^{(P)}_{i,t}$ | Percent combined score: $S^{(P)}_{i,t} = \alpha q^J_{i,t} + (1-\alpha) p_{i,t}$ |
| $R^J_{i,t}, R^V_{i,t}$ | Judges rank and fan rank (descending) |
| $R^{(R)}_{i,t}$ | Rank combined score: $R^{(R)}_{i,t} = R^J_{i,t} + R^V_{i,t}$ |
| $\xi_t$ | Slack for elimination-consistency; penalized in vote inference |
| $\lambda$ | Penalty weight on slacks in vote inference objective |

Table 1: Core notations used throughout the paper.

## 2.4 Tasks and Evaluation KPIs

Our workflow consists of four tightly coupled tasks. **Task A** reconstructs latent weekly fan vote shares $p_{i,t}$ for each active contestant by solving a season-level constrained optimization problem that enforces elimination consistency (with slack) under the documented mechanics. **Tasks B/C** use these inferred votes to replay each season under alternative aggregation rules (rank-based vs. percent-based, optionally with a bottom-two judges' save), producing counterfactual eliminations and placements. **Task D1** fits mixed-effects models to contrast the drivers of judges' evaluations versus audience support (celebrity attributes and professional partners). **Task D2** proposes an implementable voting rule and selects parameters via replay-based stress tests.

To evaluate models and rule variants consistently, we use the following KPIs and slices.

**KPI1: Elimination Hit-Rate.** On each *eligible* $(s,t)$ (season–week), we compute the predicted eliminated set $\widehat{E}_{s,t}$ under a specified aggregation scheme and compare it to the true eliminated-at-end-of-week set $E_{s,t}$ parsed from `results`. For multi-elimination weeks with $k_{s,t} > 1$, we evaluate via bottom-$k$ set match: $\mathbf{1}\{\widehat{E}_{s,t} = E_{s,t}\}$. Weeks with no elimination are excluded from single-elimination denominators. We report KPI1 separately for rank-combination and percent-combination, and aggregate by the registry-defined eras: Rank-era seasons $\{1, 2, 28\text{–}34\}$ and Percent-era seasons $\{3\text{–}27\}$. Because the Season-28 switch is an assumption, we stress-test alternative cutoffs (27/28/29).

**KPI3: Rule Divergence Rate (Rank vs. Percent).** Holding the same vote input (e.g., BL-0 proxy or inferred votes), KPI3 measures how often two aggregation schemes disagree on eliminations:

$$\text{KPI3} = \frac{\sum_{(s,t)\in\Omega} \mathbf{1}\{\widehat{E}^{\text{rank}}_{s,t} \neq \widehat{E}^{\text{percent}}_{s,t}\}}{|\Omega|}, \tag{1}$$

where $\Omega$ denotes eligible $(s,t)$. KPI3 is reported descriptively as "impact magnitude," not as an optimization objective.

**KPI2: Vote Identifiability (interval tightness).** Because votes are unobserved, vote inference is not always identifiable. For each $(s, t, i)$, we compute a feasible interval $[p_{i,t}^{\min}, p_{i,t}^{\max}]$ over fan vote shares consistent with elimination constraints (and with a near-optimal shell around the Task A objective). We define an identifiability score as

$$ID_{i,t} = \frac{1}{p_{i,t}^{\max} - p_{i,t}^{\min} + \varepsilon}, \tag{2}$$

and summarize by quantiles across contestants, weeks, and seasons. KPI2 flags weeks and contestants where multiple vote allocations explain the observed elimination equally well.

**Reproducible baseline (BL-0).** Before introducing inferred votes, we report KPI1/KPI3 under BL-0 as an audit baseline. In BL-0, the rank scheme reduces to judges' ranks only (uniform fan ranks add a constant), and the percent scheme uses judges' percent plus a uniform fan share $1/n_{\text{active}}$. BL-0 validates denominator handling and replay mechanics, but it is not used to claim audience preference estimates.

# 3 Data Processing and Analysis

## 3.1 Data Cleaning Principles and Validation

### 3.1.1 Cleaning Principles

Our preprocessing is *model-driven*: we only perform transformations required to (i) construct a reproducible season–week–contestant panel, (ii) define eligibility/denominator rules for KPI computation, and (iii) enable counterfactual replay under alternative voting schemes. We preserve the dataset's official encodings and remove their influence through an explicit *active-set* definition rather than aggressive imputation [1].

**Principle A — Respect official encodings (0 scores, N/A, irregular season length).** The official problem statement clarifies that (i) weeks with no elimination or multiple eliminations exist, (ii) `N/A` can mean either "no 4th judge" or "weeks beyond a season's actual run", and (iii) a score of 0 is recorded for celebrities after elimination (post-exit padding). We therefore treat these as *structural encodings* rather than noise [1].

**Principle B — Eligibility controls denominators (active-set construction).** A contestant is considered active in $(s, t)$ if at least one judge score exists for that week and the week does not exceed the contestant's exit week (eliminated week parsed from `results`, or an inferred withdraw week). Post-elimination zeros are retained for auditability but excluded from denominators via `active=false`.

**Principle C — Within-week aggregation only; no judge identity tracking.** Because judge indices are not persistent identities across weeks/seasons, we only use within-week aggregates (weekly total, rank, and percent) and avoid modeling judge-specific time series at the preprocessing stage [1].

**Principle D — Non-single-elimination weeks are handled explicitly.** Weeks with zero eliminations are excluded from the single-elimination consistency denominator, while multi-elimination weeks are evaluated via bottom-$k$ set matching (set-match).

### 3.1.2 Data Cleaning Operations

We implement a deterministic pipeline that reshapes the original wide-format judge-score columns into a long panel and then attaches minimal labels required for downstream inference and replay:

1. **Wide-to-long reshaping.** We reshape `weekX_judgeY_score` into one row per $(season, week, celebrity)$, and compute `all_judges_nan` and `total_judge_score = sum(judge_scores, skipna={True})` to respect varying judge counts.
2. **Exit mapping and labels.** We parse elimination week from `results` when available; otherwise, for withdrew/finished cases we infer `exit_week` as the last week with a positive total score, falling back to the last non-missing scored week. We store `exit_type` $\in$ {withdrew, eliminated, finished} and define `true_elim_flag` only for elimination exits by default.
3. **Active-set definition and anomaly flag.** We define `active = (not all_judges_nan) & (week` $\leq$ `exit_week)`. Additionally, if `total_judge_score == 0` occurs outside the inferred exit week, we flag `data_anomaly_zero_score=1` and force `active=false` to prevent padded zeros from contaminating denominators.
4. **Within-week relative measures.** For each $(season, week)$, among active contestants only, we compute `judge_rank` (1 = best) and `judge_percent = score / sum(scores)` when the denominator is positive.

### 3.1.3 Data Cleaning Outcomes and Validation

**Outcomes (artifacts and interfaces).** The output of preprocessing is a *canonical weekly panel* (`output/table/intermediate_weekly_panel.csv`) that contains eligibility/exit labels (`active`, `exit_type`, `exit_week`, `true_elim_flag`) and judge-based within-week features (`total_judge_score`, `judge_rank`, `judge_percent`). This panel is the single input interface used for all downstream KPI recomputation, vote-share inference, and replay simulation.

In addition, we generate audit/debug artifacts: (i) a week-level prediction table (`output/table/intermediate_baseline`) and (ii) a report-ready LaTeX table (`output/table/tab_baseline_consistency.tex`) plus an optional figure (`output/figure/fig_fliprate_by_season.pdf`).

**Validation (what we check, and why it is sufficient for P0).** We validate preprocessing through three audit layers:

- **(V1) Encoding-consistency checks.** We follow the official semantics for `N/A` and post-elimination zeros, preserving these encodings and excluding them from denominators via `active` [1].
- **(V2) Denominator/eligibility invariants.** `judge_percent` is computed only among active contestants within each $(season, week)$; when total score is positive, it sums to one within the active set. The explicit `data_anomaly_zero_score` flag prevents accidental inclusion of padded zeros.
- **(V3) End-to-end reproducibility via BL-0 sanity check.** On top of the canonical panel, we implement the BL-0 baseline (rank: judge-rank only; percent: judge-percent plus uniform $1/n_{\text{active}}$) and reproduce a consistent set of KPI1/KPI3 summaries across era slices. This serves as a pipeline sanity check rather than a final model claim.

**BL-0 sanity-check summary (reportable, not a final claim).** Under BL-0, the overall elimination hit-rate is 0.364 (rank) and 0.375 (percent) across 264 eligible weeks, and the rank-vs-percent flip rate is 0.038.

## 3.2 Panel Construction

Panel construction converts the wide-format score table into a long-format dataset where each record corresponds to a unique $(season, week, celebrity)$ unit. We reshape `weekX_judgeY_score` into such a long panel and compute weekly aggregates (`total_judge_score`) and within-week relative measures (`judge_rank`, `judge_percent`) among the active set. We then attach deterministic exit/eligibility labels (`exit_type`, `exit_week`, `true_elim_flag`, `active`) derived from `results` and score availability. This panel is the canonical input shared by all subsequent modules, including vote-share inference and counterfactual

replay under alternative voting schemes.

## 3.3 Empirical Findings from EDA (only what feeds models)

### 3.3.1 Data Visualization

We summarize baseline consistency and rule divergence using:

- **(L1 table)** `output/table/tab_baseline_consistency.tex`: BL-0 elimination hit-rate under rank and percent schemes (KPI1) and rule divergence (KPI3/FlipRate), with slices: Overall / Rank-era / Percent-era.
- **(L2 figure, optional)** `output/figure/fig_fliprate_by_season.pdf`: season-level visualization of where the two schemes disagree.

### 3.3.2 Key Empirical Findings

**Finding 1 — A reproducible BL-0 baseline exists and is non-trivial.** Using only judge-based information and a uniform fan proxy, BL-0 yields measurable elimination-consistency hit-rates under both rank and percent aggregation, and a non-zero flip rate between the two schemes. These quantities are used strictly as a sanity check for the pipeline and as a reference point for later vote-inference models.

**Finding 2 — Rule comparisons must be stress-tested around the season-28 cutoff.** We adopt the era split for rule-comparison slicing (Rank-era vs. Percent-era), but treat the season-28 switch as an assumption and stress-test adjacent cutoffs.

**Finding 3 — Non-standard weeks materially affect denominators.** Because the dataset includes weeks with no elimination or multiple eliminations, KPI denominators must explicitly exclude no-elimination weeks and handle multi-elimination weeks via set matching; otherwise consistency metrics can be inflated.

## 3.4 Feature Engineering

We keep feature engineering minimal and aligned with downstream modeling needs:

- **Performance signals (week-level, within active set).** `total_judge_score`, `judge_rank`, and `judge_percent` are computed per $(season, week)$ among active contestants and serve as the observed performance component when replaying elimination rules or inferring latent vote shares.
- **Explanatory covariates (contestant-level, for heterogeneity analysis).** When available, we use `ballroom_partner` (pro dancer identity), `celebrity_age_during_season`, `celebrity_industry`, and region-related fields for slicing and mixed-effects heterogeneity analyses.

## 3.5 Synthesis of Findings

In summary, our preprocessing produces a canonical season–week–contestant panel that (i) respects the official encodings of `N/A` and post-elimination zeros, (ii) enforces denominator correctness through an explicit active-set definition, and (iii) enables reproducible baseline replay consistent with our KPI definitions. Key data caveats—non-standard elimination weeks, varying judge availability, and the uncertain season-28 era cutoff—are surfaced explicitly for robustness stress tests rather than hidden by aggressive cleaning [1].

# 4 Task A: Fan Vote Inference Model

We propose a deployable, data-driven fan vote inference model. The key design principle is to **treat audience votes as a latent signal and infer vote shares from observed judges' scores and elimination outcomes under explicit rule constraints.**

## 4.1 Overview and Modeling Philosophy

**Fan votes are a latent signal and must be inferred indirectly.** In DWTS, eliminations depend on a combination of judges' scores and audience votes, yet only judges' scores and realized eliminations are observed. We therefore model weekly audience support as a latent vote-share vector over the active set. Because many vote configurations can be consistent with the same elimination outcome, the inference problem is generally underdetermined; our approach resolves this by (i) enforcing elimination-consistency constraints implied by the show's rule mechanics and (ii) selecting a stable solution via temporal smoothness regularization. Special episodes (e.g., no-elimination or irregular-format weeks) are handled explicitly through slack variables rather than forcing exact compliance, and these slacks become a diagnostic for identifiability and model mismatch.

## 4.2 Main Inference Model

**Decision variables.** For each season $s$, week $t$, and active contestant $i \in A_{s,t}$, let $p_{i,t} \geq 0$ denote the unobserved fan vote share, with $\sum_{i \in A_{s,t}} p_{i,t} = 1$. Let $J_{i,t}$ be the observed total judges' score (sum across judges with missing values skipped) and define the judges' share $q^J_{i,t} = J_{i,t} \,/\, \sum_{r \in A_{s,t}} J_{r,t}$.

**Percent-scheme combined score (primary inference target).** Under the percent-based aggregation, we define the combined score

$$S^{(P)}_{i,t} = \alpha q^J_{i,t} + (1-\alpha) p_{i,t}, \tag{3}$$

where $\alpha \in [0,1]$ controls the relative weight of judges versus fans (default $\alpha = 0.5$; later stress-tested).

**Elimination-consistency constraints with slack.** Let $E_{s,t} \subseteq A_{s,t}$ denote the observed eliminated set at the end of week $t$ with $m_{s,t} = |E_{s,t}|$. We introduce a nonnegative slack $\xi_t \geq 0$ and impose

$$S^{(P)}_{e,t} \leq S^{(P)}_{i,t} + \xi_t, \quad \forall e \in E_{s,t},\ \forall i \in A_{s,t} \setminus E_{s,t}. \tag{4}$$

When the assumed rule and data are perfectly compatible, the minimum feasible $\xi_t$ is zero; otherwise $\xi_t > 0$ quantifies the smallest violation needed to reconcile the observed outcome.

**Objective with judge-anchor prior.** To avoid degenerate "nearly-uniform" vote reconstructions, we add a judge-anchor prior that pulls fan vote shares toward judges' score shares in the absence of strong contradictory evidence. For each season $s$, we solve:

$$\begin{aligned} \min_{\{p_{i,t}\},\{\xi_t\}} \quad & \underbrace{\sum_{t=2}^{T_s} \sum_{i \in A_{s,t} \cap A_{s,t-1}} (p_{i,t} - p_{i,t-1})^2}_{\text{Temporal smoothness}} + \lambda_1 \underbrace{\sum_{t=1}^{T_s} \sum_{i \in A_{s,t}} (p_{i,t} - q^J_{i,t})^2}_{\text{Judge anchor prior}} + \lambda_2 \underbrace{\sum_{t=1}^{T_s} \xi_t^2}_{\text{Violation penalty}} \\ \text{s.t.} \quad & \sum_{i \in A_{s,t}} p_{i,t} = 1, \quad p_{i,t} \geq 0, \quad \xi_t \geq 0, \qquad \forall t, \\ & S^{(P)}_{e,t} \leq S^{(P)}_{k,t} + \xi_t, \quad \forall t,\ \forall e \in E_{s,t},\ \forall k \in A_{s,t} \setminus E_{s,t}. \end{aligned} \tag{5}$$

**Remark (objective form).** Equation (5) is a convex quadratic program (QP) solved independently for each

season. The smoothness term is computed only for contestants active in consecutive weeks ($A_{s,t} \cap A_{s,t-1}$), while the anchor and slack penalties apply to all active contestants and weeks. Hyperparameters $(\lambda_1, \lambda_2)$ are selected by replay-based diagnostics (Section 4.4).

**Algorithm 1** Season-wise fan vote inference via constrained QP

**Require:** Weekly panel for season $s$: active sets $A_{s,t}$, judges' totals $J_{i,t}$, eliminated sets $E_{s,t}$; hyperparameters $\alpha, \lambda_1, \lambda_2$.

**Ensure:** Point estimates $\hat{p}_{i,t}$ and slack diagnostics $\hat{\xi}_t$ for all eligible weeks.

1: Compute judges' share for each week $t$:

$$q^J_{i,t} = \frac{J_{i,t}}{\sum_{r \in A_{s,t}} J_{r,t}}$$

2: Define decision variables $\{p_{i,t}\}_{i \in A_{s,t}}$ with constraints:

$$p_{i,t} \geq 0, \quad \sum_{i \in A_{s,t}} p_{i,t} = 1$$

3: For each week $t$, define percent-based composite score $S^{(P)}_{i,t}$ and slack variable $\xi_t$:

$$S^{(P)}_{i,t} = \alpha q^J_{i,t} + (1-\alpha) p_{i,t}, \quad \xi_t \geq 0$$

4: Add elimination-consistency constraints:

$$S^{(P)}_{e,t} \leq S^{(P)}_{i,t} + \xi_t \quad \forall e \in E_{s,t},\ i \in A_{s,t} \setminus E_{s,t}$$

5: Solve the quadratic program (QP) to minimize the objective function:

$$\mathcal{L} = \sum_{t=2}^{T_s} \sum_{i \in A_{s,t} \cap A_{s,t-1}} (p_{i,t} - p_{i,t-1})^2 + \lambda_1 \sum_{t=1}^{T_s} \sum_{i \in A_{s,t}} (p_{i,t} - q^J_{i,t})^2 + \lambda_2 \sum_{t=1}^{T_s} \xi_t^2$$

6: Output $\hat{p}_{i,t}$ (inferred vote shares) and $\hat{\xi}_t$ (slack diagnostics); flag weeks with large $\hat{\xi}_t$ as potential special-episode mismatches.

## 4.3 Uncertainty Quantification and Sensitivity Analysis

**Uncertainty is intrinsic because the inverse problem is not fully identifiable.** We report three complementary uncertainty summaries: (i) **Feasible intervals:** for each $(i,t)$, compute the minimum and maximum $p_{i,t}$ attainable under the same constraints (yielding an interval width as an identifiability proxy, KPI2); (ii) **Elimination margin:** quantify how much perturbation to $\hat{p}_{i,t}$ is required to flip the predicted eliminated set under the replay engine, providing a week-level stability score; and (iii) **Slack diagnostics:** $\xi_t$ indicates weeks where observed outcomes cannot be matched under the assumed mechanics without violation; these weeks are flagged for special-episode handling and for sensitivity analysis over the rule-switch season and $\alpha, \lambda_1, \lambda_2$.

### 4.3.1 KPI2: Vote Identifiability (interval tightness)

**KPI2: Vote Identifiability (interval tightness).** The KPI registry defines vote identifiability by the tightness of a feasible interval for each $(s,t,i)$ under an explicit elimination mechanism. In Task A, we instantiate KPI2 under the **percent-based rule** with the fan vote **share** variable $p_{i,t}$, and set $\varepsilon = 10^{-6}$ to avoid singularities. After solving the season-wise inference QP and obtaining $(\hat{p}, \hat{\xi})$ with optimal objective value $\mathrm{OPT}_s$, we

compute a **near-optimal feasible interval** for each active contestant-week pair by solving two auxiliary profile problems:

$$p_{i,t}^{\min} = \min p_{i,t}, \qquad p_{i,t}^{\max} = \max p_{i,t},$$

subject to the original probability and elimination-consistency constraints, plus a tolerance shell that keeps solutions comparable to the fitted trajectory:

$$\mathcal{L}(p, \xi) \le (1 + \delta_{\text{obj}})\text{OPT}_s, \qquad \xi_\tau \le \hat{\xi}_\tau + \delta_\xi \ \ (\forall \tau),$$

where $\mathcal{L}(p, \xi)$ is the Task-A objective (smoothness + judge-anchor + slack penalty). The interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ is mapped to an identifiability score

$$\text{ID}_{i,t} = \frac{1}{w_{i,t} + \varepsilon}.$$

We summarize $\text{ID}_{i,t}$ by median/quantiles across contestants and weeks and aggregate by season and by early/mid/late phases; in implementation we compute KPI2 on selected slices (e.g., late-phase weeks or controversial cases) to control computational cost.

**Profile optimization formulation.** For a fixed season $s$, let $T_s$ be the number of weeks. Define decision variables

$$\mathbf{p} = \{p_{j,\tau} : j \in A_{s,\tau},\ \tau = 1, \dots, T_s\}, \qquad \boldsymbol{\xi} = \{\xi_\tau : \tau = 1, \dots, T_s\}.$$

For each week $\tau$, the percent-rule combined score is

$$S_{j,\tau}^{(P)} = \alpha q_{j,\tau}^{J} + (1 - \alpha) p_{j,\tau}.$$

**Base constraints.** We impose simplex constraints for each week:

$$p_{j,\tau} \ge 0, \quad \forall j \in A_{s,\tau}, \forall \tau, \tag{6}$$

$$\sum_{j \in A_{s,\tau}} p_{j,\tau} = 1, \quad \forall \tau. \tag{7}$$

Slacks are nonnegative:

$$\xi_\tau \ge 0, \qquad \forall \tau. \tag{8}$$

Elimination-consistency constraints (same as Task A) are

$$S_{e,\tau}^{(P)} \le S_{k,\tau}^{(P)} + \xi_\tau, \quad \forall \tau,\ \forall e \in E_{s,\tau},\ \forall k \in A_{s,\tau} \setminus E_{s,\tau}. \tag{9}$$

**Near-optimal shell (to prevent uninformative, overly wide intervals).** Let the Task-A objective be

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) = \sum_{\tau=2}^{T_s} \sum_{j \in A_{s,\tau} \cap A_{s,\tau-1}} (p_{j,\tau} - p_{j,\tau-1})^2 + \lambda_1 \sum_{\tau=1}^{T_s} \sum_{j \in A_{s,\tau}} (p_{j,\tau} - q_{j,\tau}^{J})^2 + \lambda_2 \sum_{\tau=1}^{T_s} \xi_\tau^2. \tag{10}$$

After solving the main inference QP, we obtain $(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$. To compute intervals around the fitted explanation, we add:

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) \le (1 + \delta_{\text{obj}})\, \text{OPT}_s, \tag{11}$$

$$\xi_\tau \le \hat{\xi}_\tau + \delta_\xi, \qquad \forall \tau. \tag{12}$$

In implementation, (11) can be written as a convex SOC constraint by noting that $\mathcal{L}$ is a sum of squares:

$$\left\| \begin{bmatrix} \{p_{j,\tau} - p_{j,\tau-1}\}_{\tau=2..T_s,\ j \in A_{s,\tau} \cap A_{s,\tau-1}} \\ \{\sqrt{\lambda_1}\,(p_{j,\tau} - q_{j,\tau}^{J})\}_{\tau=1..T_s,\ j \in A_{s,\tau}} \\ \{\sqrt{\lambda_2}\,\xi_\tau\}_{\tau=1..T_s} \end{bmatrix} \right\|_2 \le \sqrt{(1 + \delta_{\text{obj}})\, \text{OPT}_s}.$$

This SOC form is directly supported by standard convex solvers (e.g., via CVX frameworks).

**Profile-min and profile-max.** For each active pair $(i,t)$ with $i \in A_{s,t}$, define:

$$p_{i,t}^{\min} = \min_{\mathbf{p},\boldsymbol{\xi}} \; p_{i,t} \quad \text{s.t. } (6)--(9),\ (11)--(12), \tag{13}$$

and

$$p_{i,t}^{\max} = \max_{\mathbf{p},\boldsymbol{\xi}} \; p_{i,t} \quad \text{s.t. } (6)--(9),\ (11)--(12). \tag{14}$$

We then set the interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $\mathrm{ID}_{i,t} = 1/(w_{i,t} + \varepsilon)$.

**Algorithm 2** KPI2 computation via profile optimization (per season)

**Require:** Season $s$ weekly panel: active sets $A_{s,t}$, eliminated sets $E_{s,t}$, judges' shares $q_{i,t}^{J}$; main-model hyperparameters $\alpha, \lambda_1, \lambda_2$; KPI2 params $\varepsilon, \delta_{\mathrm{obj}}, \delta_{\xi}$; a slice selector $\mathcal{S}$ (set of $(i,t)$ pairs to evaluate).

**Ensure:** Interval bounds $\{p_{i,t}^{\min}, p_{i,t}^{\max}\}$ and identifiability scores $\{\mathrm{ID}_{i,t}\}$ for $(i,t) \in \mathcal{S}$.

1: **Solve main inference QP** for season $s$ to obtain $(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$ and $\mathrm{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$.
2: **Define common constraints**: simplex constraints (7)–(9) and slack nonnegativity (8).
3: **Add KPI2 shells**: objective shell (11) (or its SOC form) and slack cap (12).
4: Choose evaluation slice $\mathcal{S}$ (e.g., late-phase weeks and/or controversial cases) to control computation.
5: **for** each $(i,t) \in \mathcal{S}$ **do**
6: Solve profile-min optimization (13) to get $p_{i,t}^{\min}$.
7: Solve profile-max optimization (14) to get $p_{i,t}^{\max}$.
8: Compute width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $\mathrm{ID}_{i,t} = 1/(w_{i,t} + \varepsilon)$.
9: Sanity check: verify $p_{i,t}^{\min} \le \hat{p}_{i,t} \le p_{i,t}^{\max}$; if violated, align constraints with the main QP.
10: **end for**
11: Aggregate $\mathrm{ID}_{i,t}$ by week/season/phase using medians and quantiles as defined in the KPI registry.

## 4.4 Hyperparameters, tuning, and implementation details

**Hyperparameters and defaults.** Task A uses $\alpha$ (judge weight in the percent rule), $\lambda_1$ (strength of the judge-anchor prior), $\lambda_2$ (penalty on constraint violations via $\xi_t$), and the KPI2 parameters $\varepsilon, \delta_{\mathrm{obj}}, \delta_{\xi}$. Unless otherwise stated, we set $\alpha = 0.5$ as the nominal DWTS percent weighting and treat $\alpha$ as a sensitivity knob in later stress tests.

We tune $(\lambda_1, \lambda_2)$ by replay-based diagnostics. Increasing $\lambda_2$ enforces stricter elimination-consistency (smaller $\xi_t$) at the cost of reduced feasibility for irregular-format weeks; decreasing $\lambda_2$ allows the model to absorb format mismatches but may underfit eliminations. Increasing $\lambda_1$ pulls inferred votes toward judges' score shares and helps avoid near-uniform solutions, while excessively large $\lambda_1$ may suppress genuine judge–fan disagreements that are evidenced by the observed eliminations. Practically, we tune $(\lambda_1, \lambda_2)$ on a season-by-season basis by balancing (i) the frequency and magnitude of nonzero slacks $\hat{\xi}_t$, (ii) the stability of inferred top-$k$ vote shares across adjacent weeks, and (iii) downstream replay fidelity in Task B.

**Special episodes and multi-elimination weeks.** Our formulation supports (a) multi-elimination weeks by allowing $|E_{s,t}| > 1$ and enforcing the same pairwise inequalities against all non-eliminated contestants, and (b) non-elimination or irregular weeks by allowing $E_{s,t} = \varnothing$ (no elimination constraints) while still solving the smoothness objective; such weeks can also be explicitly flagged and assessed via slack and sensitivity tests. If a season contains weeks whose outcomes cannot be reconciled with the assumed mechanics, the model will surface this via persistently large $\hat{\xi}_t$; these weeks are treated as format exceptions rather than forcing ad hoc

re-labeling.

**Solver and numerical stability.** The main inference problem is a convex QP and can be solved season-wise using standard QP solvers. The KPI2 shell can be implemented either directly as a quadratic constraint or in SOC form; the latter is solver-friendly in CVX frameworks. To avoid numerical issues, we recommend (i) normalizing judges' scores into shares $q^J$ (already done), (ii) using $\varepsilon = 10^{-6}$ for identifiability scoring, and (iii) keeping $\delta_{\text{obj}}$ small (e.g., 1–5%) so profile intervals remain comparable to the fitted explanation.

# 5 Task B & C: Rule Reply, Volting System, and Counterfactuals

Following the official rule definitions, we perform season–week counterfactual replays under (i) rank-based aggregation and (ii) percent-based aggregation, using the same weekly active set and deterministic eligibility labels to ensure a fair comparison. As a reproducible sanity-check baseline (BL-0), we use judge-only information with a uniform fan proxy and report elimination hit-rate (KPI1) and the rule divergence rate (KPI3/FlipRate). This chapter summarizes cross-season differences (Section 5.1), explains divergences via controversial case studies (Section 5.2), evaluates the incremental impact of bottom-two and judges' save (Section 5.3), and synthesizes implications (Section 5.4).

## 5.1 Rank vs. Percent Across Seasons

Results in this section are first reported under BL-0 as a reproducible sanity check; Task A inferred votes are used in subsequent replays.

### 5.1.1 Replay protocol and eligibility

**Goal.** This section quantifies how much the season outcomes depend on the aggregation rule by replaying each season under (i) percent-based aggregation and (ii) rank-based aggregation while holding inputs constant. For every season–week, we restrict attention to the *active set* $A_{s,t}$ and use judges' information (e.g., total judges' score or its normalized share) together with the inferred fan vote shares $\hat{p}_{i,t}$ from Task A. Given an elimination count $m_{s,t}$ (including $m_{s,t} = 0$ for non-elimination weeks and $m_{s,t} > 1$ for multi-elimination weeks), the replay engine deterministically selects the bottom $m_{s,t}$ contestants under the chosen rule and updates the active set for the next week.

**Percent rule.** Under percent-based aggregation, the combined score is

$$S_{i,t}^{(P)}(\alpha) = \alpha q_{i,t}^{J} + (1-\alpha)\hat{p}_{i,t},$$

and the replay eliminates the $m_{s,t}$ contestants with the smallest $S_{i,t}^{(P)}$.

**Rank rule.** Under rank-based aggregation, we compute judges' rank $R_{i,t}^{J}$ from total judges' score $J_{i,t}$ (descending), and vote rank $R_{i,t}^{V}$ from $\hat{p}_{i,t}$ (descending). The combined rank is

$$R_{i,t}^{(R)} = R_{i,t}^{J} + R_{i,t}^{V},$$

and the replay eliminates the $m_{s,t}$ contestants with the largest $R_{i,t}^{(R)}$ (worst combined ranks).

**Tie-handling.** To keep comparisons reproducible, ties at the elimination cutoff are resolved deterministically. For percent scores, ties are broken by higher judges' share $q^J$ (survives), and if still tied by a stable identifier order. For rank scores, we use standard competition ranking for tied values and apply the same deterministic tie-breaker when selecting the bottom $m_{s,t}$. All ties are resolved deterministically (details in Appendix X) to ensure replay reproducibility.

### 5.1.2 Cross-season summary metrics

We compute KPI1/KPI3 over eligible (season, week) pairs, excluding weeks with no elimination or irregular/multi-elimination formats per our slice definitions. **KPI1 (elimination hit-rate)** measures whether a replayed rule correctly identifies the observed eliminated set in eligible weeks. **KPI3 (FlipRate)** measures how often the predicted eliminated set differs between percent-based and rank-based replays under the same inputs. Interpreting these metrics together separates *fidelity* (does a rule reproduce observed eliminations) from *rule sensitivity* (does changing the rule change who leaves).

In addition to season-level aggregates, we report an era-sliced view aligned with the show's documented format changes. This prevents Simpson's-paradox-style conclusions where one rule appears "better" simply because it is evaluated disproportionately in seasons that used it.

### 5.1.3 Era cutoff assumption and stress-test plan

Because the exact season boundary of the return to rank-based aggregation may affect interpretation, we treat the era cutoff as an explicit assumption and stress-test adjacent cutoffs (e.g., shifting the boundary by $\pm 1$ season). Conclusions that remain stable under this perturbation are reported as robust, while boundary-sensitive findings are explicitly qualified.

## 5.2 Controversial Contestants: Case Studies

We select cases from seasons with high FlipRate and large judge-fan discordance, ensuring that each case demonstrates a distinct mechanism.

**Purpose.** Aggregate FlipRate identifies *where* rules diverge; case studies explain *why* the divergence occurs and how it translates into public controversy. We analyze a small set of representative contestants (e.g. Jerry Rice and Bobby Bones) whose outcomes are commonly discussed in the context of judge-fan disagreement. For each case, we follow a fixed protocol:

- **Observed trajectory:** week-by-week judges' rank, survival, and final placement.
- **Counterfactual replays:** percent vs. rank under the same inferred $\hat{p}_{i,t}$, and (when applicable) with/without a judges' save module.
- **Mechanism attribution:** identify the minimal driver of outcome change (e.g., persistent advantage in vote rank that dominates under rank aggregation, or "rank compression" effects when many contestants cluster in judges' scores).
- **Uncertainty qualification:** if the week-level vote interval is wide (KPI2) or replay margins are small, we report the case as sensitive and avoid over-claiming.

This structure ensures that each narrative claim is backed by a reproducible replay trace rather than anecdotal reasoning.

## 5.3 Bottom-two and Judges' Save Mechanism

Some later seasons introduce a two-stage mechanism: (i) identify a bottom subset using the vote aggregation rule, then (ii) select the eliminated contestant from that subset via judges' save. To isolate the incremental effect of this change, we implement a modular judges' save layer that can be toggled on top of either percent-based or rank-based aggregation.

**Module definition.** In a week with a bottom-two format, the replay first forms a bottom-two set $B_{s,t}$ using the chosen aggregation rule. The saved contestant is the one with higher judges' score $J_{i,t}$ (equivalently higher $q^J_{i,t}$), and the other is eliminated. This construction cleanly separates *how the bottom set is formed* from *how the final elimination is chosen*, enabling controlled counterfactual comparisons.

**Evaluation questions.** We quantify (a) how often the judges' save reverses the elimination implied by votes alone, (b) whether the save disproportionately protects contestants with strong judges' scores but weak fan support, and (c) how the mechanism affects overall rule divergence and late-season stability. These results connect directly to management decisions about perceived fairness versus entertainment value.

## 5.4 Synthesis of Findings from Rules

We synthesize the rule replay evidence into four actionable takeaways.

**(1) Rule choice matters most when judges and fans disagree.** Weeks with large judge–fan rank discordance concentrate a disproportionate share of rule flips; in such weeks, percent-based aggregation tends to preserve score magnitude information, while rank-based aggregation amplifies relative ordering.

**(2) Divergence is not uniform across the season.** Rule sensitivity often increases in late-season weeks when the field is smaller and elimination margins narrow; this motivates reporting uncertainty and replay margins alongside any counterfactual claim.

**(3) Judges' save changes the source of authority.** A bottom-two + judges' save mechanism shifts the final elimination decision toward judges, reducing the ability of extreme popularity alone to determine outcomes in the bottom set—at the cost of potentially weakening audience agency.

**(4) Implication for redesign.** These patterns motivate a new aggregation system (Task D2) that preserves interpretability and replay stability while explicitly controlling how much popularity can overwhelm technical performance. The next chapters use the same replay engine to evaluate proposed parameters under robustness and stress tests.

# 6 Task D-1: Impact Analysis: Pro Dancers & Celebrity Attributes

## 6.1 Outcome definitions

To disentangle how outcomes are shaped by professional partners versus celebrity attributes, we model judges' evaluations and inferred fan preferences as two distinct but related outcomes. For comparability across seasons and weeks, we work with *shares* rather than raw scores. Let $A_{s,t}$ denote the active set in season $s$, week $t$.

**Judges' outcome.** Define the judges' score share

$$q^J_{i,t} = \frac{J_{i,t}}{\sum_{r \in A_{s,t}} J_{r,t}}, \tag{15}$$

and use a logit transform with a small $\epsilon$ to avoid boundary issues:

$$y^J_{i,t} = \log \frac{q^J_{i,t} + \epsilon}{1 - q^J_{i,t} + \epsilon}. \tag{16}$$

**Fans' outcome.** Let Task A provide the inferred vote share $p_{i,t}$ (denoted $q^V_{i,t}$). We similarly define

$$y^V_{i,t} = \log \frac{p_{i,t} + \epsilon}{1 - p_{i,t} + \epsilon}. \tag{17}$$

We adopt share + logit for cross-season comparability and interpret coefficients on the log-odds scale.

## 6.2 Modeling results: separating "who judges reward" vs. "who fans support"

We fit two mixed-effects models on the season–week panel to attribute variation to (i) celebrity covariates and (ii) professional dancer partners.

**Model 1: Judge Model.**

$$y^J_{i,t} = \beta_0 + \beta_X^\top X_i + \beta_T f(t) + FE_s + u_i + v_{d(i)} + \epsilon_{i,t}, \tag{18}$$

where $X_i$ collects time-invariant celebrity attributes (e.g., age, occupation indicators), $f(t)$ captures week trend, $FE_s$ is a season fixed effect, $u_i \sim \mathcal{N}(0, \sigma_u^2)$ is a celebrity random effect, and $v_{d(i)} \sim \mathcal{N}(0, \sigma_v^2)$ is a professional-dancer random effect. A key hypothesis test is $H_0 : \sigma_v^2 = 0$ (no pro-dancer effect on judges).

**Model 2: Vote Model.** To model inferred fan support while allowing mediation through judges' performance, we include $y^J_{i,t}$:

$$y^V_{i,t} = \gamma_0 + \gamma_J y^J_{i,t} + \gamma_X^\top X_i + \gamma_T f(t) + FE_s + a_i + b_{d(i)} + \eta_{i,t}, \tag{19}$$

with $a_i \sim \mathcal{N}(0, \sigma_a^2)$ and $b_{d(i)} \sim \mathcal{N}(0, \sigma_b^2)$ capturing celebrity and pro-dancer random effects on fan preferences, respectively. The coefficient $\gamma_J$ measures the transmission strength from judges to fans.

**Comparing judges vs. fans.** For each shared attribute $k$, define the preference gap

$$\Delta_k = \gamma_k - \beta_k. \tag{20}$$

$\Delta_k > 0$ indicates fans prefer the attribute more than judges do; $\Delta_k < 0$ indicates the opposite. We report $\Delta_k$ with uncertainty bands (via bootstrap) whenever vote inference uncertainty is non-negligible.

**Quantifying professional-dancer impact (magnitude, not just significance).** We quantify the fraction of outcome variance attributable to professional dancers using ICC-style ratios:

$$ICC^J_{pro} = \frac{\sigma_v^2}{\sigma_v^2 + \sigma_u^2 + \sigma^2}, \qquad ICC^V_{pro} = \frac{\sigma_b^2}{\sigma_b^2 + \sigma_a^2 + \sigma_\eta^2}. \tag{21}$$

We treat BLUP rankings as descriptive summaries of the fitted random effects rather than causal estimates to compare whether the same pros are "good for judges" and "good for fans."

**Uncertainty propagation.** Because fan votes are inferred, we propagate uncertainty either by (i) bootstrap re-estimation of votes and refitting the Vote Model, or (ii) weighted regression using the interval width from Task A as an uncertainty proxy; we report confidence intervals accordingly.

# 7 Task D-2: New Rule Design and Stress Testing

## 7.1 Design Goals and Principles

Our replay analysis shows that outcome sensitivity concentrates in weeks where judges' evaluations and audience preferences diverge, and that purely rank-based aggregation can amplify small ordering differences into large outcome changes. A redesigned system should therefore: (i) preserve audience agency while (ii) preventing extreme popularity from overwhelming technical performance, (iii) remain interpretable to viewers, (iv) use few tunable parameters, and (v) be robust to vote-estimation uncertainty and minor format variations.

We operationalize these goals with measurable criteria: high replay fidelity on eligible weeks (KPI1), reduced instability under perturbations (stability under uncertainty), and transparent parameterization for management control.

## 7.2 Proposed aggregation rule: Judge Safeguard + Smoothed Rank (JS-SR)

We propose an implementable rule that balances technical merit and audience agency by introducing a *Safeguard Zone*. Pure percent blending can allow extreme popularity to dominate, while pure rank aggregation can discard meaningful score margins. JS-SR protects top technical performers while letting fan preference decide outcomes within the remaining "battleground" set.

**Step 1: Safeguard threshold.** For each week $t$ with active set $A_{s,t}$, let $\tau \in (0,1)$ be a management-controlled percentile (e.g., $\tau = 0.30$ meaning "top 30% by judges"). Define the week-specific threshold on raw total judges' scores $J_{i,t}$:

$$\text{Threshold}_{s,t} = \text{Percentile}\big(\{J_{i,t}\}_{i\in A_{s,t}},\ 1-\tau\big). \tag{22}$$

The safeguarded set is

$$\text{SafeSet}_{s,t} = \{i \in A_{s,t} \mid J_{i,t} \geq \text{Threshold}_{s,t}\}. \tag{23}$$

Contestants in $\text{SafeSet}_{s,t}$ are immune from elimination in week $t$.

**Step 2: Battleground scoring and elimination.** Let $D_{s,t} = A_{s,t} \setminus \text{SafeSet}_{s,t}$ be the battleground set. For $i \in D_{s,t}$, define the combined score using judges' share $q^J_{i,t}$ and inferred fan share $p_{i,t}$:

$$S^{(JS)}_{i,t} = w_J\, q^J_{i,t} + (1-w_J)\, p_{i,t}, \qquad w_J \in [0,1]. \tag{24}$$

We eliminate the $m_{s,t}$ contestants with the smallest $S^{(JS)}_{i,t}$ among $D_{s,t}$ (deterministic tie-handling).

**Algorithm 3** Algorithm 3: JS-SR replay under $(\tau, w_J)$

**Require:** Weekly active sets $A_{s,t}$, raw judges totals $J_{i,t}$, judges shares $q^J_{i,t}$, inferred fan shares $p_{i,t}$, elimination counts $m_{s,t}$, parameters $(\tau, w_J)$.

**Ensure:** Eliminated sets $\tilde{E}^{(JS)}_{s,t}$ and final placements under JS-SR.

**for** $t = 1$ to $T_s$ **do**
  Restrict to active set $A_{s,t}$.
  **if** $m_{s,t} = 0$ **then**
    $\tilde{E}^{(JS)}_{s,t} \leftarrow \varnothing$; continue to next week.
  **end if**
  Compute $\text{Threshold}_{s,t}$ as the $(1-\tau)$ percentile of $\{J_{i,t}\}_{i\in A_{s,t}}$.
  Form safe set:

$$\text{SafeSet}_{s,t} = \{i \in A_{s,t} : J_{i,t} \geq \text{Threshold}_{s,t}\}$$

  Form elimination candidate pool: $D_{s,t} = A_{s,t} \setminus \text{SafeSet}_{s,t}$.
  For each $i \in D_{s,t}$, compute combined score:

$$S^{(JS)}_{i,t} = w_J q^J_{i,t} + (1-w_J) p_{i,t}$$

  Eliminate the $m_{s,t}$ contestants with smallest $S^{(JS)}_{i,t}$ within $D_{s,t}$ (deterministic tie-handling applied).
  Update next week's active set: $A_{s,t+1} \leftarrow A_{s,t} \setminus \tilde{E}^{(JS)}_{s,t}$.
**end for**

### 7.3 Stress Testing and Simulation Evidence via Replay

We evaluate the proposed rule using a season–week replay protocol identical to Chapters 4–5, ensuring fair comparisons across rule variants. Stress tests sweep $(\tau, w_J)$ over a predefined grid and propagate vote uncertainty by sampling plausible vote scenarios within Task A uncertainty bands (or bootstrap reconstructions). For each configuration, we report KPI1 on eligible weeks, divergence relative to historical rules, and stability metrics quantifying how often small parameter or vote perturbations change (i) the eliminated set and (ii) final placements. Recommended parameters are selected by multi-metric trade-off analysis, prioritizing robust performance rather than tuning to any single season.

## 8 Conclusion

In this study, we model *Dancing with the Stars* as an inverse problem in which judges' scores and eliminations are observed while audience vote totals are withheld. Task A provides a convex inference framework to reconstruct weekly fan vote shares consistent with observed eliminations and to quantify identifiability and uncertainty. Building on these inferred votes, Tasks B and C implement season–week replays that isolate the impact of aggregation mechanics (percent versus rank) and optional bottom-two judges' save, revealing where rule changes most strongly affect outcomes. Task D1 then separates the drivers of judges' evaluations and fan support using mixed-effects structures that attribute variation to celebrity attributes and professional partners. Finally, Task D2 proposes a simple, interpretable rule that power-transforms fan vote shares before percent blending, and evaluates it through stress tests over parameters and vote-uncertainty perturbations.

Collectively, our pipeline produces an auditable chain from data to mechanism interpretation to actionable rule recommendations, while explicitly flagging weeks and seasons where non-identifiability limits the strength of conclusions. Future work could incorporate richer behavioral signals (e.g., external popularity proxies) and explore alternative mechanism designs under the same replay-and-stress-test framework.

## 9 Memo to Management

**To: Show Management**
**Subject: Recommendation for a More Stable and Transparent Voting System**

Our analyses indicate that controversy risk concentrates in weeks where judges' evaluations and audience preferences diverge. To preserve audience agency while preventing extreme popularity from overwhelming technical performance, we recommend piloting *Judge Safeguard + Smoothed Rank (JS-SR)*.

Under JS-SR, each week designates a *Safeguard Zone*: contestants above a judges-score percentile threshold (top $\tau$) are immune from elimination. The remaining contestants compete in a "battleground" where elimination is determined by a transparent weighted blend of judges' share and fan share:

$$S_{i,t}^{(JS)} = w_J \, q_{i,t}^{J} + (1 - w_J) \, p_{i,t}.$$

Management controls two intuitive knobs: $\tau$ (how many top technical performers are protected) and $w_J$ (judge–fan balance within the battleground). We recommend selecting $(\tau, w_J)$ from a robust region identified by historical replays and uncertainty stress tests, then deploying JS-SR in a one-season "shadow mode" computed in parallel with the broadcast rule to quantify differences in eliminations, finalists, and winner outcomes before adoption.

Operationally, we recommend a one-season *shadow deployment* in which the proposed rule is computed in parallel with the broadcast rule to quantify differences in eliminations, finalists, and winner outcomes, accompanied by a clear public explanation of tie-handling and the role of any judges' save mechanism. This provides a controlled, auditable path to improving fairness perception and reducing outcome volatility without sacrificing interpretability.

# References

[1] COMAP, “2026 MCM problem C: Data with the stars (problem statement pdf),” MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C.pdf

[2] ——, “2026 MCM problem C dataset (problem_c_data.csv),” MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C_Data.csv

# A Appendix: Data, Algorithms, and Reproducibility

# B Complete Files for Problem B

See 2625838 for repository tree and executiomn steps.

# Report on Use of AI

**Team Control Number:** 2625838
**Problem:** C

## AI Tools Used

| AI Tool | Version / Model | Primary Purpose |
|---|---|---|
| OpenAI ChatGPT | GPT-5.2 | Language polishing, structural refinement, and clarification of written explanations |
| OpenAI ChatGPT | GPT-5.2 | Brainstorming ideas for modeling approaches, algorithm designs, and solution strategies |
| OpenAI ChatGPT | GPT-5.2 | Generating preliminary code drafts for data processing, graph producing and simulation, later reviewed and modified by the team |
| OpenAI ChatGPT | GPT-5.2 | Synthesis of the explanatory documents, work orders, template tickets for internal team Delivery |
| OpenAI ChatGPT | GPT-5.2 | Giving Advice on engineered Document Management and Version Control best practices |
| Perplexity | Deep Research | Assistance on searching reference for relevant techniques |
| Github Copilot | — | Enhance productivity while maintaining full control over the code quality and integrity |

## Query/Response Record

**ChatGPT Query 1:**
**Input:** We just finished our setting of *dynamic_files*, please refer to the *first_stage_modeling.md* and *context_pack_for_ai.md* in our repository, and search for authorative and highly credible supporting literature on topics of convex optimization/scalable solving, underdetermined inversion/regularization, ranking/rule differences, compositional data, simulation and sensitivity
**Output:** See full answers in chatgpt_1_output.md.

**ChatGPT Query 2:**
**Input:** Please help us check our draft of our frame of the report as I listed, and give suggestions on improving the structure. (For the sake of clarity and format consistency, see the conplete conversation record in chatgpt_2_input_and_output.md)
**Output:** Your framework is usable, but it is recommended to revise Chapters 4–7 into a four-part structure: "Fan Vote Inference → Rule Replay Comparison → Impact Factors → New Mechanism Proposal". This way, the table of contents can directly correspond to the four task points of the problem at a glance, making it easier for reviewers to read and more like "answering the problem". (see the conplete conversation record in chatgpt_2_input_and_output.md)

**ChatGPT Query 3:**
**Input:** Please follow the work order ticket to finish the triage stage of data cleaning suggestions. (For the sake of clarity and format consistency, see the conplete conversation record in chatgpt_3_input_and_output.md)
**Output:** The season-week-contestant weekly panel has been successfully built in accordance with the P0 standards of dynamic_files/data_agent_triage_ticket.md, and the specified files have been delivered (including reproducible BL-0 baseline and consistency KPIs). See the conplete conversation record in chatgpt_3_input_and_output.md.

**ChatGPT Query 4:**
**Input:** Wait, why is the generated .pdf file completely unchanged? Could there be an issue with how we implemented the rules? Also, where in the code you provided do we need to modify the paths for importing and exporting files? (See the conplete conversation record in chatgpt_4_input_and_output.md)
**Output:** The most common reason you see "no changes" in the PDF is not an error in the rules, but that the path/file you're opening is not the newly generated one. Additionally, our BL-0 baseline inherently results in a FlipRate very close to 0, making the plot appear "largely unchanged". The plot you uploaded itself shows FlipRate fluctuating around 0 (with a very small coordinate range)—this is an expected outcome under BL-0. (see the conplete conversation record in chatgpt_4_input_and_output.md)

**ChatGPT Query 5:**
**Input:** Currently, our .py code files are located at 2026_MCM/src/LI_play_data/*.py, and we expect the generated charts to be saved to 2026_MCM/output/fig (or 2026_MCM/output/tab for tables, and the required data to be saved to 2026_MCM/output/data_cleaned). I have several questions:
(a) Do we need to manually create the output/fig and output/tab directories in advance?
(b) If not, do we need to modify the paths in the Python files, and how? (We do not plan to use .ipynb files.)
(c) After modifying the paths in the files, how should I run the script on a Windows computer to generate the new files? Will the newly generated files overwrite the existing ones after running the script?
**Output:** Understood: You currently place the entry script at 2026_MCM/src/LI_play_data/*.py and expect outputs to be saved to:

- 2026_MCM/output/fig for figures
- 2026_MCM/output/tab for tables
- 2026_MCM/output/data_cleaned for cleaned data

Below are step-by-step answers to your 4 questions, along with a minimal modification approach (only adjust paths/directory creation without changing core logic). (see the conplete conversation record in chatgpt_5_input_and_output.md)

**ChatGPT Query 6:**
**Input:** You are the "Data Phase Master Controller + Standard Auditor" for my MCM/ICM project. Our problem-solving approach focuses on optimization/strategy/simulation, and we will not conduct general EDA. Please work only based on the files I upload; do not fabricate any data, results, fields, units, or problem requirements. If uncertain, mark it as "Not Provided/To Be Confirmed" and provide minimal supplementary information requirements ($\geq$ 3 items) along with the safest default handling plan.(See complete prompt in chatgpt_6_input_and_output.md)
**Output:** See full outputs in chatgpt_6_output.md.

**ChatGPT Query 7:**
**Input:** What should we do regarding the previous conversation? Because the investigation file is non-authoritative; the files we feed to the agent do not include the investigation—it is only used for us to check if our framing/spec agent has any important omissions. Now clearly tell me how to use the information you provided above (e.g., 3. Place the Data Agent Triage work order in... file / place it separately in the conversation as a prompt).
**Output:** In line with your current constraints (the investigation file is non-authoritative and will not be fed to the Data Agent; it is only used to check for omissions in the framing/spec), you can use my previous output as two types of materials: (1) an executable work order for the Data Agent + (2) minimal standard patches for backfilling dynamic_files. The specific operations are as follows (step by step, no need to modify the investigation file further). (see the conplete conversation record in chatgpt_7_input_and_output.md)

**ChatGPT Query 8:**
**Input:** 'Handling of "Withdrew": Should it be "exclude after inferring the exit week" or "exclude the entire season from the consistency denominator"? Consistency denominator: Exclude no-elimination weeks, use set-match for multi-elimination weeks (is this acceptable?).' What is the best way to handle this? Is it part of our cleaning standards? If we only add this one-sentence patch, will it cause huge problems in subsequent data processing?
**Output:** This is part of the cleaning standards, and it must be implemented in the kpi_registry; otherwise, two types of "huge problems" will occur later:
1. During the Data phase, everyone will handle withdrew/no-elimination/multi-elimination according to their own intuition, leading to inconsistent KPI denominators $\rightarrow$ irreproducible results;
2. In your writing, you mention "consistency", but when reviewers ask "how is the denominator calculated", you cannot answer or give inconsistent responses.
Therefore: Adding a patch is not a risk; on the contrary, it reduces risk. The real risk is "not clarifying it clearly". (see the conplete conversation record in chatgpt_8_input_and_output.md)

**ChatGPT Query 9:**
**Input:** Please check the setting of kpis from you and me and give me suggestions on how to deal with that. (See the conplete conversation record in chatgpt_9_input_and_output.md)
**Output:** When you said "I removed many KPIs", the reason is straightforward: I was defining the P0/Triage metrics framework—only retaining metrics that are 100% implementable with the core research brief + existing official CSV data, runnable within 2–4 hours, and require no data fabrication due to missing information. Several metrics in your original framework fall under the P1/P2 categories (requiring vote estimation models, additional assumptions, external data, or threshold calibration). To prevent the data team from being forced to "guess/impute/fabricate" data during the P0 phase, I temporarily excluded them from the minimal version of the authoritative KPI table. (see the conplete conversation record in chatgpt_9_input_and_output.md)

**ChatGPT Query 10:**
**Input:** Now let's go through the materials and prompt to be handed over to the Data Agent for triage: ... (see the conplete conversation record in chatgpt_10_input_and_output.md) Could you check if there are any issues with this prompt?
**Output:** Your prompt is on the right track overall (with clear authoritative standards, ticket-driven execution and product delivery paths), but I recommend making 3 minor revisions; otherwise, the Data Agent may still make mistakes or deviate from the requirements during implementation: (see the conplete conversation record in chatgpt_10_input_and_output.md)

**ChatGPT Query 11:**
**Input:** Please refer to chatgpt_11_input_and_output.md for complete conversation record.
**Output:** See full outputs in chatgpt_11_input_and_output.md

**ChatGPT Query 12:**
**Input:** Our team members have completed the following work on the data: ... (see the conplete conversation record in chatgpt_12_input_and_output.md) I think it may not be wise to continue spending time on this data, and our team member's modeling approach has already been finalized. Should we first complete the in-depth data cleaning and then compare with our team members

to select the optimal approach, or directly proceed to the core work of our writing team after aligning our dynamic documents?
**Output:** Your intuition is correct: it is unwise to continue spending time on "general feature engineering/general cleaning". Now that your core modeling approach is finalized (QP inversion for fan share + rule playback simulation + mixed effects), the optimal workflow is: First align and freeze the "specifications and data contract" → immediately integrate Model V1 to complete a closed-loop run → then conduct "model-driven deep cleaning/risk diagnosis (Top-3 risks)" → finally implement visualization and writing. Reason: Your model essentially relies on consistent specifications for "active set/ground truth eliminations/within-week normalization (percents)/N/A weeks and post-elimination zero-score encoding"; these are explicitly problematic areas in the problem statement (N/A weeks, zero scores, no/multiple eliminations, uncertainty about the return to rank-based aggregation in Season 28). (see the conplete conversation record in chatgpt_12_input_and_output.md)

**ChatGPT Query 13:**
**Input:** So let me sort out what we need to do now: First step: Align our specifications. You mentioned freezing and sticking to our core data pipeline, but actually I don't think this is necessary. Because after my teammate makes modifications, they will put their work in their own folder on GitHub, and mine will go into my folder. Even though we're in the same GitHub repository, we can cross-check each other's work later—it'll be like mutual supplementation. Second step: Integrating Model V1. I don't quite understand this part—does V1 refer to a code file (not a DOCX document) that will make the model we've designed actually run? Is that the right understanding? Then third step: Conduct in-depth data cleaning, followed by visualization and finalizing the writing. Is this the correct workflow? I also have another question: Earlier, we identified two risks of inconsistent specifications, right? Well, I suggest we fix these specification inconsistencies first—preferably by modifying the code. I'll send you the code, and could you guide me on exactly what needs to be changed?
**Output:** Your summary is mostly correct. Let me "calibrate" the key points, then directly guide you on **modifying the code first to unify specifications** (I've already reviewed the 'p0_triage_build_weekly_panel.py' file you sent this time). (see complete conversation record in chatgpt_13_input_and_output.md)

**ChatGPT Query 14:**
**Input:** No rush, let's first sort out what we currently have:
1. We have the complete problem statement.
2. We have the preliminary data produced by our data team member just now.
3. We have the preliminary model developed by our modeling team member.
Please let me know what other files are needed and I will provide them.

Therefore, I think our workflow should be as follows:
1. First, complete the construction of the framework.
2. Then, provide you with our modeling spec, and then see which parts we can complete (my initial idea is that we can first finish the "Problem Restatement", "Key Assumptions", "Core Variables", and a bit of the beginning of the model; please supplement anything that is missing).
3. We will investigate the existing files and write the parts that can be included in the article.
**Output:** Your workflow plan is correct, and given that you "will not perform additional data cleaning in this phase", this is the most stable and rework-free way to proceed.
Below, I will refine your proposed 3 steps into an "executable checklist + the minimum set of files you need to provide + the writing blocks I can produce". By following this, you can directly start producing the main text.(see complete conversation record in chatgpt_14_input_and_output.md)

# Human Contribution Declaration

We affirm that:

1. All final modeling decisions, algorithm designs, and strategic choices were made by the team members.
2. All mathematical formulations and assumptions were independently derived or verified by the team.
3. All code included in this project was reviewed, tested, and fully understood by the team.
4. AI tools were used only for brainstorming, drafting, language refinement, and preliminary coding assistance.
5. The submitted work represents our own understanding and intellectual contribution.

Date: January 22, 2026